MIT | Academy of Engineering
(An Autonomous Institute Affiliated to Savitribai Phule University)

ASSIGNMENT 2

Differentiate Between HTML and HTML5 in Terms of Features and Usage, with Semantic vs. Non-Semantic Structure and Web Page Demonstration.

Student Details

Student Name	PRN	Class / Branch	Subject
Umang Ramteke	202401100068	B.Tech (IT)	Web Technology (WT)

Aim

To differentiate between traditional Hypertext Markup Language (HTML 4.01/XHTML) and modern HTML5 in terms of architectural features, syntax, multimedia capabilities, client storage, and real-world usage; to illustrate the qualitative contrast between Good HTML (semantic tags, document hierarchy, web accessibility, and clean separation of concerns) and Bad HTML (non-semantic *div-soup*, inline presentation styling, deprecated tags, and lack of form validation); and to demonstrate these core concepts through a fully interactive comparison web page.

Objectives

Technical Objectives

1. **Architectural Evolution:** Analyze the transition from SGML-based document definitions (DTD in HTML 4.01) to the streamlined, plugin-independent HTML5 Living Standard specified by W3C and WHATWG.
2. **Semantic Tagging & Document Outlining:** Examine native structural semantic elements (<*header*>, <*nav*>, <*main*>, <*article*>, <*section*>, <*aside*>, <*footer*>, <*figure*>, <*time*>) versus generic non-semantic containers (<*div*>, <*span*>).
3. **Native Multimedia & Graphics Architecture:** Investigate how HTML5 replaces vulnerable third-party proprietary plugins (Flash, Silverlight) with standard <*video*>, <*audio*>, <*canvas*>, and inline <*svg*> elements.
4. **Declarative Form Controls & Validation:** Contrast modern HTML5 input types (*email*, *tel*, *url*, *number*, *date*, *range*) and native constraint validation attributes (*required*, *pattern*, *min*, *max*) against fragile JavaScript-dependent text inputs.
5. **Client-Side Storage & Performance APIs:** Evaluate high-capacity Web Storage (*localStorage*, *sessionStorage*, *IndexedDB*) versus legacy 4 KB cookies, and examine concurrency via the Web Workers API.

Functional and Demonstration Objectives

1. **Code Quality Categorization:** Define explicit, measurable parameters distinguishing Good Semantic HTML from Bad Non-Semantic HTML across accessibility, search indexing, maintainability, and responsiveness.
2. **Interactive Web Demonstration:** Develop and deploy an interactive web page (*demo_html_vs_html5.html*) that renders Good and Bad HTML implementations side-by-side with live code inspection and accessibility trees.

Theory & Key Concepts

A. Evolution of Hypertext Markup Language (HTML to HTML5)

Hypertext Markup Language was created by Tim Berners-Lee in 1991 to share scientific documents across computer networks. As the web evolved from static hyperlinked documents into an interactive application platform, the limitations of HTML 4.01 (standardized in 1999) became apparent. HTML 4.01 relied on complex SGML Document Type Definitions (DTDs), lacked native multimedia support, and had no standard semantics for page layout, forcing developers to construct complex nested structures of un-semantic <*div*> tags with arbitrary class names. The World Wide Web Consortium (W3C) and the Web Hypertext Application Technology Working Group (WHATWG) collaborated to engineer HTML5. Standardized in 2014 and continuously maintained as a Living Standard, HTML5 re-architected the web into a rich application runtime featuring native audio/video, offline storage, hardware-accelerated 2D/3D graphics, standardized parsing algorithms, and dedicated semantic layout elements.

B. Comprehensive Feature Differentiation: HTML vs. HTML5

The fundamental architectural differences between classic HTML (HTML 4.01/XHTML) and modern HTML5 are contrasted below across eight key engineering dimensions:

Dimension	Classic HTML (HTML 4.01 / XHTML)	Modern HTML5 (Living Standard)
1. DOCTYPE	Long, SGML-based, strict/transitional DTD: *<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" ...>*	Simple, clean, minimal, and case-insensitive: *<!DOCTYPE html>*
2. Semantic Tags	No layout semantics; relies entirely on generic <*div*> and <*span*> tags with arbitrary classes.	Native semantic tags: <*header*>, <*nav*>, <*main*>, <*article*>, <*section*>, <*aside*>, <*footer*>, <*figure*>, <*time*>.
3. Multimedia	No native audio/video. Required third-party plugins (Adobe Flash, Silverlight, QuickTime).	Native hardware-accelerated <*video*> and <*audio*> tags with subtitle track synchronization (<*track*>).
4. Graphics	Dynamic graphics required external plugins (Flash, Java Applets) or server-rendered images.	Native 2D drawing via <*canvas*> API, declarative vector graphics via inline <*svg*>, and WebGL.
5. Storage	Limited to HTTP Cookies (approx. 4 KB per domain), transmitted with every HTTP request header.	High-capacity Web Storage: *localStorage* (5-10 MB), *sessionStorage*, and transactional *IndexedDB*.
6. Forms	Basic input types (*text*, *password*); validation required extensive custom JavaScript.	Rich typed inputs (*email*, *tel*, *date*, *number*) with declarative constraints (*required*, *pattern*).
7. Mobile	Desktop-centric; lacked native viewport metadata for dynamic mobile scaling.	Mobile-first architecture powered by <*meta name="viewport"*> and fluid responsive design.
8. Threads & APIs	Single-threaded execution; heavy scripts froze the UI. Polling via stateless AJAX.	True multi-threaded **Web Workers**, full-duplex **WebSockets**, and standardized Geolocation.

C. Good HTML vs. Bad HTML: Architectural & Semantic Analysis

The quality of an HTML document is determined not merely by whether a visual browser renders the layout without errors, but by whether the underlying document structure accurately reflects its conceptual meaning, accessibility hierarchy, and algorithmic parsability.

1. Semantic Hierarchy vs. 'Div-Soup' Anti-Pattern

Good HTML utilizes semantic container tags (<*header*>, <*nav*>, <*main*>, <*article*>, <*section*>, <*aside*>, <*footer*>) to create an explicit document outline. Assistive technologies (screen readers like NVDA and VoiceOver) translate these landmarks into navigable waypoints, allowing visually impaired users to jump directly to specific sections. In contrast, **Bad HTML** produces 'div-soup'—a deeply nested maze of generic <*div*> and <*span*> elements with arbitrary class names (e.g., <*div class="nav-bar"*>). To a screen reader or search engine web crawler, div-soup represents a flat, unnavigable wall of plain text without landmarks or structural significance.

2. Modern Form Controls & Native Validation vs. Generic Text Fields

In **Good HTML**, form elements are programmatically paired with <*label for="id"*> tags, enabling screen readers to vocalize field purposes and allowing users to click label text to focus inputs. HTML5 introduces typed inputs (*email*, *tel*, *number*, *date*) that automatically trigger optimized virtual keyboards on mobile devices and enforce declarative validation rules (*required*, *pattern*, *min*, *max*) natively in the browser without writing brittle JavaScript. Conversely, **Bad HTML** uses untyped <*input type="text"*> fields lacking label associations, relies on unstyled floating text, and uses fake submit buttons (e.g., <*div onclick="validate()"*>) that cannot be operated using keyboard Tab or Enter keys.

3. Separation of Concerns vs. Deprecated Presentational Tags

HTML5 strictly enforces the separation of content structure (HTML) from visual presentation (CSS) and interactivity (JavaScript). **Bad HTML** violates this principle by embedding obsolete presentational elements (<*font*>, <*center*>, <*marquee*>, <*big*>), inline styles (*style="color:red; font-size:12px;"*), and table-based layout grids (<*table width="100%"*>). This bloated markup degrades page loading speeds, destroys responsive adaptability on mobile screens, and drastically inflates maintenance costs.

4. Media Embeds & Web Accessibility (WCAG 2.1)

Good HTML guarantees web accessibility by mandating meaningful alternative text (*alt="..."*) for informative images, explicit figure captions (<*figure*><*figcaption*>), native video controls with subtitles (<*track kind="captions"*>), and proper heading hierarchies (a single <*h1*> logically nested down to <*h6*>). In **Bad HTML**, images lack *alt* attributes or use redundant filenames, heading tags are picked solely for their default font sizes rather than semantic importance, and media elements rely on deprecated embed objects that crash on mobile devices.

D. Comparative Code Demonstrations: Semantic HTML5 vs. Non-Semantic HTML

Good HTML: Semantic HTML5 Architecture

```
<!-- Clean, self-describing semantic document
-->
<header>
  <h1>EcoTech Blog</h1>
  <nav aria-label="Main">
    <ul><li><a href="#news">News</a></li></ul>
  </nav>
</header>
<main>
  <article>
    <header>
      <h2>Clean Solar 2026</h2>
      <time datetime="2026-10-15">Oct 15</time>
    </header>
    <p>Efficiency reached <mark>32%</mark>.</p>
    <figure>
      <img src="cell.webp" alt="Photovoltaic
cell">
      <figcaption>Fig 1: Absorption
rate</figcaption>
    </figure>
  </article>
  <form action="/sub" method="POST">
    <label for="em">Email Address:</label>
    <input type="email" id="em" required>
    <button type="submit">Subscribe</button>
  </form>
</main>
<footer><small>© 2026 EcoTech
Inc.</small></footer>
```

Bad HTML: Legacy Non-Semantic Div-Soup

```
<!-- Div soup: Inaccessible, zero meaning -->
<div id="header">
  <div class="title">EcoTech Blog</div>
  <div id="menu">
    <div onclick="goNews()">News</div>
  </div>
</div>
<div id="content-wrapper">
  <div class="post-box">
    <div class="heading">Clean Solar 2026</div>
    <div class="date">Oct 15, 2026</div>
    <div>Efficiency reached <b>32%</b>.</div>
    <table width="100%"><tr><td align="center">
      <img src="cell.gif"><br/>
      <font size="1">Fig 1: Absorption</font>
    </td></tr></table>
  </div>
  <div class="form-box">
    Email: <input type="text" id="txtEmail">
    <span class="btn"
onclick="chk()">Subscribe</span>
  </div>
</div>
<div id="foot"><font size="1">© 2026
EcoTech</font></div>
```

E. Architecture of the Interactive Demonstration Web Page

To experimentally validate the theoretical differentiation between HTML and HTML5, an interactive dual-architecture web application (**demo_html_vs_html5.html**) was developed. Key functional components include:

- **Side-by-Side Live View:** Renders Good HTML (semantic HTML5 layout) and Bad HTML (div-soup with inline scripts) concurrently to highlight how deceptive visual rendering hides catastrophic structural flaws.
- **Accessibility & DOM Tree Visualizer:** Exposes the Accessibility Tree, demonstrating how screen readers recognize semantic landmarks (*banner*, *navigation*, *main*, *contentinfo*) versus flat generic text in div-soup.

Applications / Practical Use Cases

1. **Search Engine Optimization (SEO) & Knowledge Graph Extraction:** Modern search crawlers (Googlebot, Bingbot) leverage semantic HTML5 tags (<*article*>, <*header*>, <*time*>, <*nav*>) to parse document hierarchy, extract rich snippets, and index primary content while discarding boilerplate.
2. **Web Accessibility & Assistive Technology (WCAG 2.1 Compliance):** International accessibility mandates (such as Section 508 and the European Accessibility Act) legally require digital accessibility. Semantic HTML5 tags construct ARIA landmarks automatically without fragile manual ARIA attributes.
3. **Progressive Web Applications (PWAs) & Mobile Browsing:** HTML5 features including responsive viewport meta tags, offline Web Storage (*localStorage*, *IndexedDB*), native audio/video playback, and Service Workers enable web apps to function seamlessly across mobile devices.
4. **Enterprise Codebase Maintainability:** Standardized semantic HTML drastically reduces reliance on bloated CSS classes and bespoke JavaScript validation scripts, lowering development overhead, debugging time, and bandwidth consumption across enterprise portals.

Software Requirements

Component	Specification / Tool Used
Operating System	Windows 10 / 11, macOS, or Linux (64-bit architecture)
Markup & Standards	HTML5 (WHATWG Living Standard), HTML 4.01 Strict/Transitional DTD
Styling Engine	Modern CSS3 (CSS Flexbox, CSS Grid, Custom Properties, Media Queries)
Supported Web Browsers	Google Chrome (v110+), Microsoft Edge (v110+), Mozilla Firefox (v110+)
Code Editor / IDE	Visual Studio Code (VS Code) with HTML5 & Prettier Extensions
HTML Validation Tool	W3C Nu HTML Validator (https://validator.w3.org/nu/)

Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 (Dual-core, 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 or higher
RAM (Memory)	4 GB RAM	8 GB RAM or higher
Hard Disk Space	500 MB free storage for local code and cache	2 GB SSD free disk space
Display Resolution	1366 x 768 pixels	1920 x 1080 (Full HD)
Input Devices	Standard Keyboard and Mouse / Touchpad	Keyboard with Tab navigation and pointing device
Network Interface	Required for web font loading and deployment	Broadband Internet connection (5 Mbps+)

Code & Demonstration Links :-

Source Code Repository (GitHub):

https://github.com/umangramteke/labour-connect

Live Demonstration Web Page (Netlify):

https://labour-connect1.netlify.app/demo_html_vs_html5.html

Local Project Demonstration File:

assignmnet 5/demo_html_vs_html5.html (or assignment-2/index.html)

Conclusion

The comparative study and experimental implementation conclusively demonstrate that HTML5 is not merely an incremental syntactic update to HTML 4.01, but a fundamental architectural transformation of the World Wide Web from a static hyperlinked document delivery system into an advanced, universal application runtime. By introducing native semantic landmarks (<*header*>, <*nav*>, <*main*>, <*article*>, <*footer*>), hardware-accelerated multimedia (<*video*>, <*audio*>), declarative form constraint validation, and robust client storage APIs, HTML5 completely resolves the structural and technical deficiencies of legacy markup. Furthermore, the practical comparison between Good HTML and Bad HTML clearly proves that adopting semantic structure is essential for screen reader accessibility (WCAG 2.1 compliance), search engine optimization (SEO), cross-platform device responsiveness, and long-term codebase maintainability. The accompanying demonstration web page provides clear empirical proof of these advantages in a live browser environment.